



# CPSC 100

## Computational Thinking

### **Application of CT: Algorithms**

**Instructor: Dr. Firas Moosvi**  
**Department of Computer Science**  
**University of British Columbia**

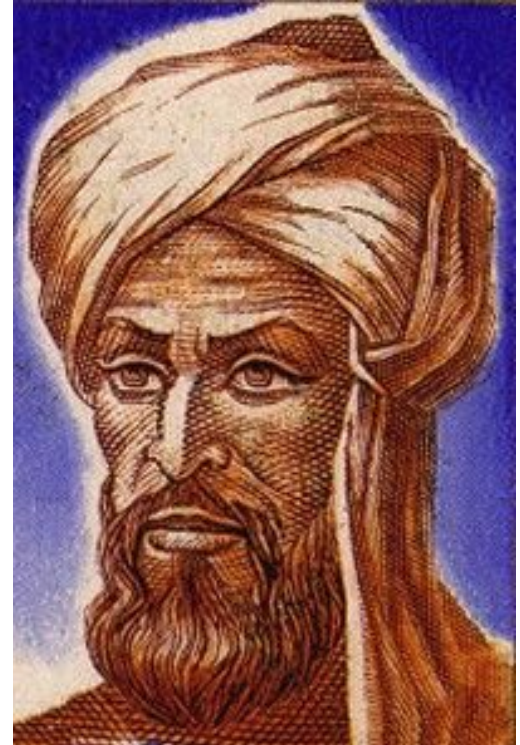
# Slides for Pre-reading

**Slides with yellow borders  
will not be covered in  
class, but are still testable  
content - you should  
review this before class.**

# Algorithms

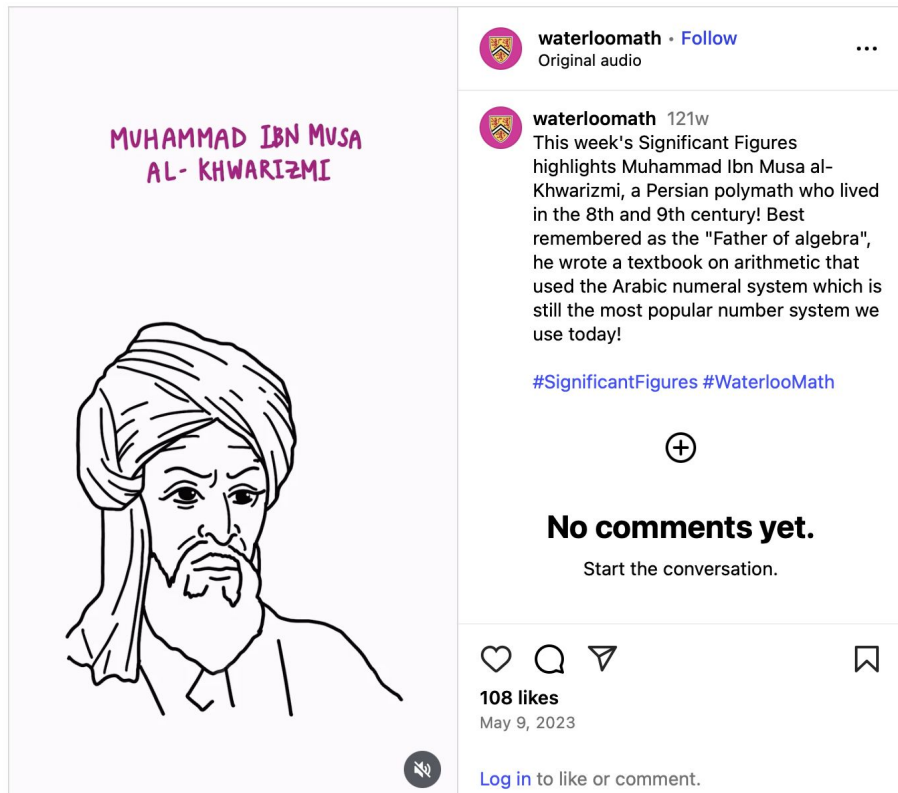
Muhammad ibn Musa al-Khwarizmi  
(*Algorithmi*)

- Persian mathematician around 800 CE
- Discussed how to formulate mathematical procedures



# Algorithms - Check out this Reel

Instagram



# To sort, you create an *algorithm*

An *algorithm* is a precise, systematic method for producing a specified result.

Your sorting algorithms used:

- Decomposition
- Abstraction
- Evaluation

# To sort, you create an *algorithm*

An *algorithm* is a precise, systematic method for producing a specified result.

Your sorting algorithms used:

- **Decomposition:** breaking down the problem into smaller tasks you could solve

# To sort, you create an *algorithm*

An *algorithm* is a precise, systematic method for producing a specified result.

Your sorting algorithms used:

- **Abstraction:** describing the solution in a general way that's applicable no matter what order the cards are in initially



# To sort, you create an *algorithm*

An *algorithm* is a precise, systematic method for producing a specified result.

Your sorting algorithms used:

- **Evaluation:** Assessing a solution to a problem and using that information again on new problems.

# Course Admin



# Agenda

- Applying Computational Thinking (CT)
  - Sorting Class activity
  - Discussion
- Introduction to Algorithms

# Learning Goals



# Learning Goals

After this lecture, you should be able to:

- Apply CT subskills to design and execute a structured solution
- Explain/define the concept of Algorithms
  - Describe its relevance to CT and where it originated from
- Define the concepts of *decomposition*, *abstraction* and *evaluation* in relation to an algorithm

# Class Discussion

# Participation Question

**Did you come up with a list of steps to sort cards?**

Successful: What worked?

Unsuccessful: Where did it fail ?

Logistics

Counter Example



# Algorithms





# Algorithms

An ***algorithm*** describes a sequence of steps that is:



# Algorithms

An ***algorithm*** describes a sequence of steps that is:

## 1. Unambiguous

- No “assumptions” are required to execute the algorithm
- The algorithm uses precise instructions



# Algorithms

An ***algorithm*** describes a sequence of steps that is:

## 1. Unambiguous

- No “assumptions” are required to execute the algorithm
- The algorithm uses precise instructions

## 2. Executable

- The algorithm can be carried out in practice



# Algorithms

An ***algorithm*** describes a sequence of steps that is:

## 1. Unambiguous

- No “assumptions” are required to execute the algorithm
- The algorithm uses precise instructions

## 2. Executable

- The algorithm can be carried out in practice

## 3. Terminating

- The algorithm will eventually come to an end, or halt



To sort, you create an *algorithm*



# To sort, you create an *algorithm*

An *algorithm* is a precise, systematic method for producing a specified result.



# To sort, you create an *algorithm*

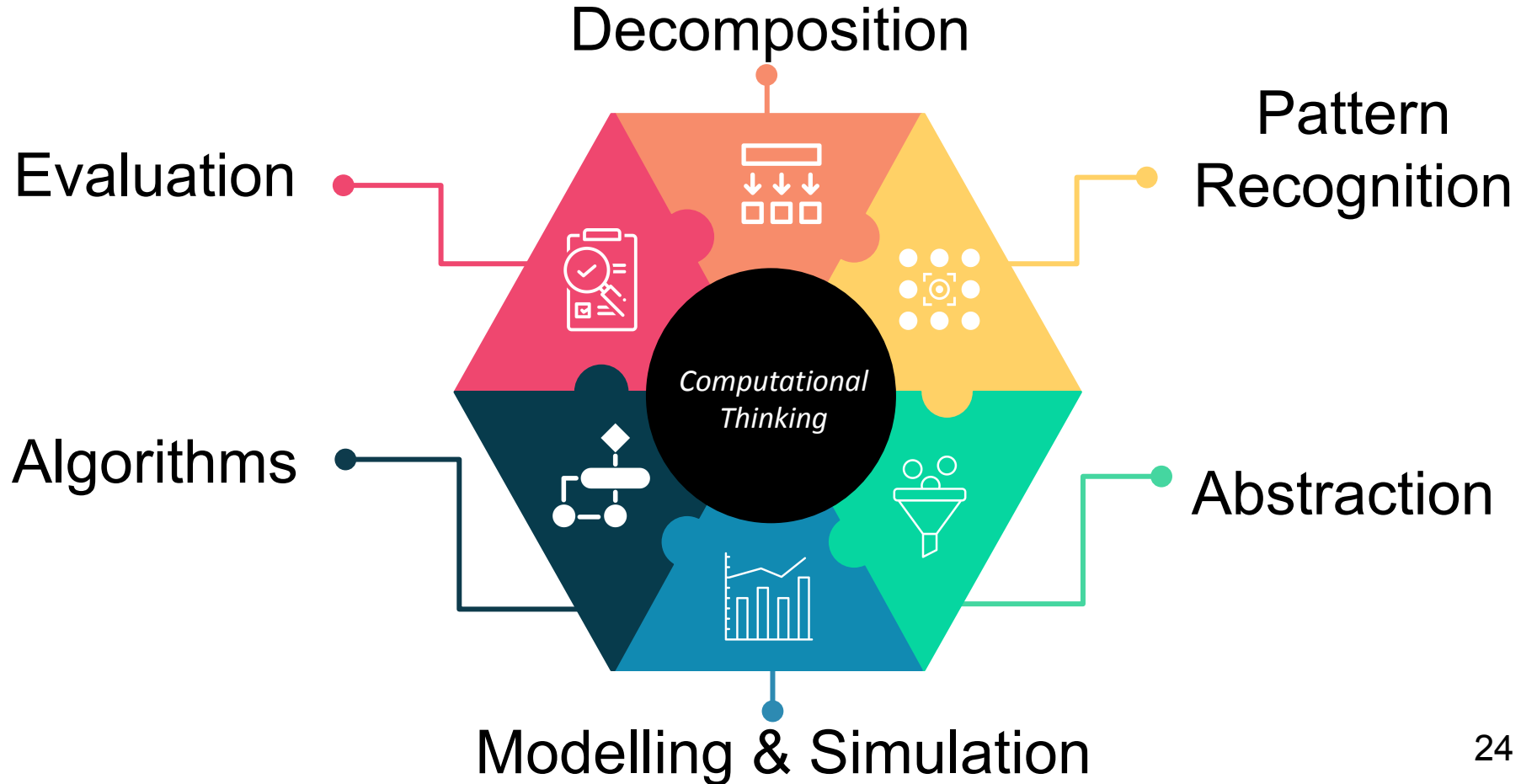
An *algorithm* is a precise, systematic method for producing a specified result.

Your sorting algorithms used:

- Decomposition\*
- Abstraction\*
- Evaluation\*

*\*definitions in the appendix*

# Computational Thinking Skills







## DECOMPOSITION

Breaking down problems into smaller, easier parts.



## PATTERN RECOGNITION

Using patterns in information to solve problems.



## ABSTRACTION

Finding information that is useful and taking away any information that is unhelpful.



## MODELLING AND SIMULATION

Trying out different solutions or tracing the path of information to solve problems.



## ALGORITHMS

Creating a set of instructions for solving a problem or completing a task.



## EVALUATION

Assessing a solution to a problem and using that information again on new problems.

# Decomposition



## DECOMPOSITION

Breaking down problems into smaller, easier parts.

**"If you can't solve a problem, then there is an easier problem you can solve: find it."**

**- George Polya**

# Abstraction



## ABSTRACTION

Finding information that is useful and taking away any information that is unhelpful.



"The **most important** and high-level thought process in computational thinking is the **abstraction process**."

- Jeannette Wing

"The **most important** and high-level thought process in computational thinking is the **abstraction process**.

Abstraction is used in **defining patterns, generalizing from instances, and parameterization**. It is used to **let one object stand for many**."

- Jeannette Wing

# Evaluation



## EVALUATION

Assessing a solution to a problem and using that information again on new problems.

"We are drowning in information, while starving for wisdom. The world henceforth will be run by **synthesizers**, people able to **put together the right information at the right time**, think critically about it, and make important choices wisely."

- Edward O. Wilson



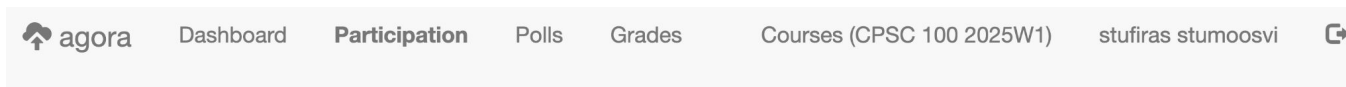
# Clicker Questions (using Agora)



# Agora: Discussions and Polling

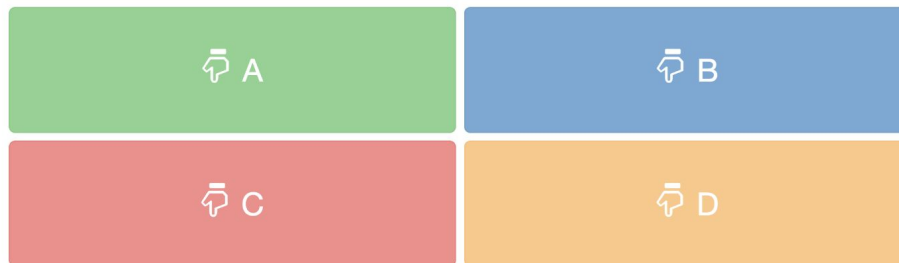
Page: [agora.students.cs.ubc.ca](https://agora.students.cs.ubc.ca) (login with your CWL)

Enroll code: **conformitygate**



CPSC 100 2025W1

Participation Points: + (Phantom)



Message Board

Lecture not started.

**Q: In the context of algorithms, \_\_\_\_\_ is a way of describing the solution in a general manner.**

- A. Decomposition
- B. Evaluation
- C. Computational Thinking
- D. Abstraction
- E. Encapsulation



**Q: In the context of algorithms, \_\_\_\_\_ is a way of describing the solution in a general manner.**

- A. Decomposition
- B. Evaluation
- C. Computational Thinking
- D. Abstraction**
- E. Encapsulation



# Class Activity

# Recall: Sort the Cards

Imagine a robot must arrange a set of cards in ascending order (Ace to King, Same suit).

The robot can only follow your instructions.

Task [Groups of 3-4]

**Create a clear set of steps/instructions to sort the card**





# Recall: Robot Constraints

The robot has some limitations!

The robot can only:

- “**Recognize**” suits (♣, ♠, ♦, ♥) and these cards: 2-10, A, K, Q, J
- “**Look**” at one card at a time
- “**Pick up**” one card at a time
- “**Move**” one card to the beginning, middle,
- “**Remember**” the location and value of one previous card (not necessarily the last one it looked at)
- “**Compare**” the numeric value of two cards and choose the higher or lower



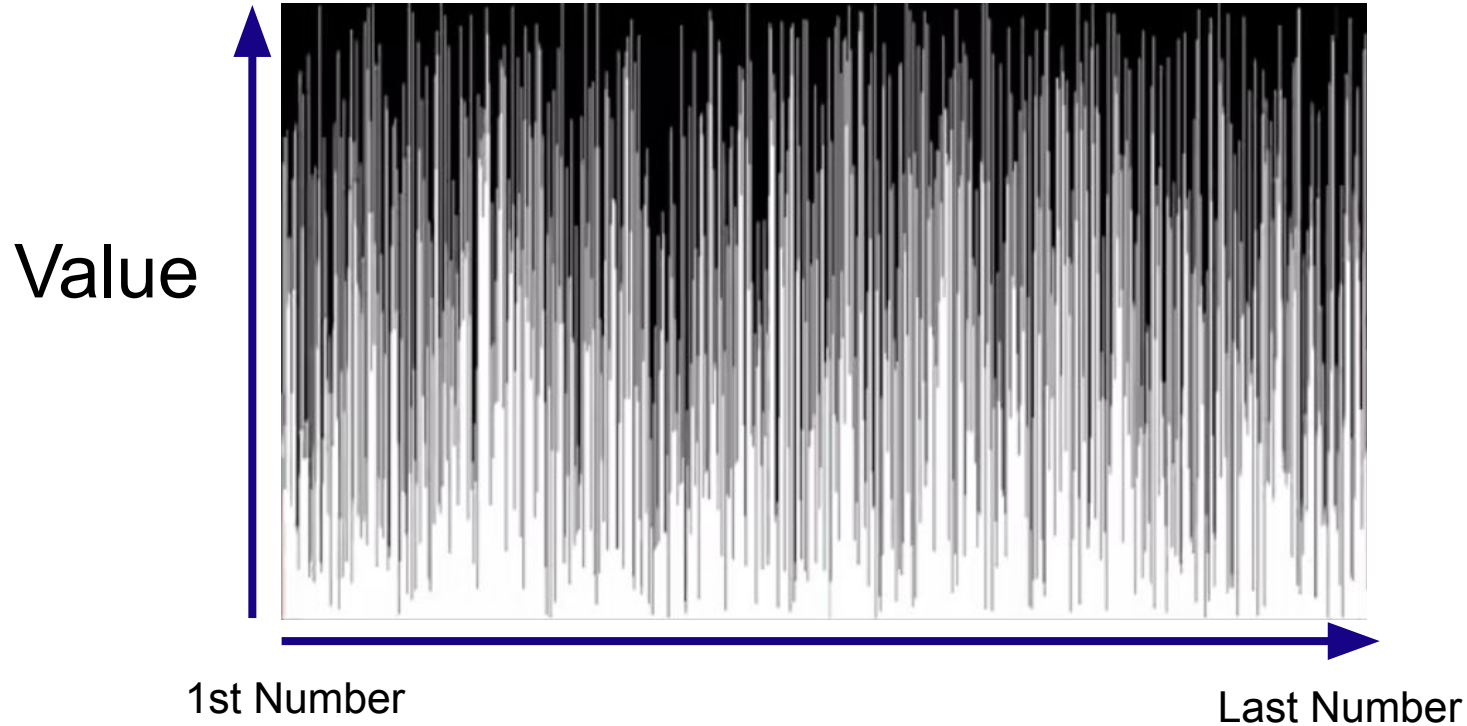
# Let's simplify things a bit...

- Let's say you want to sort a 100,000 randomly generated numbers from smallest to largest
- That's a LOT of numbers!



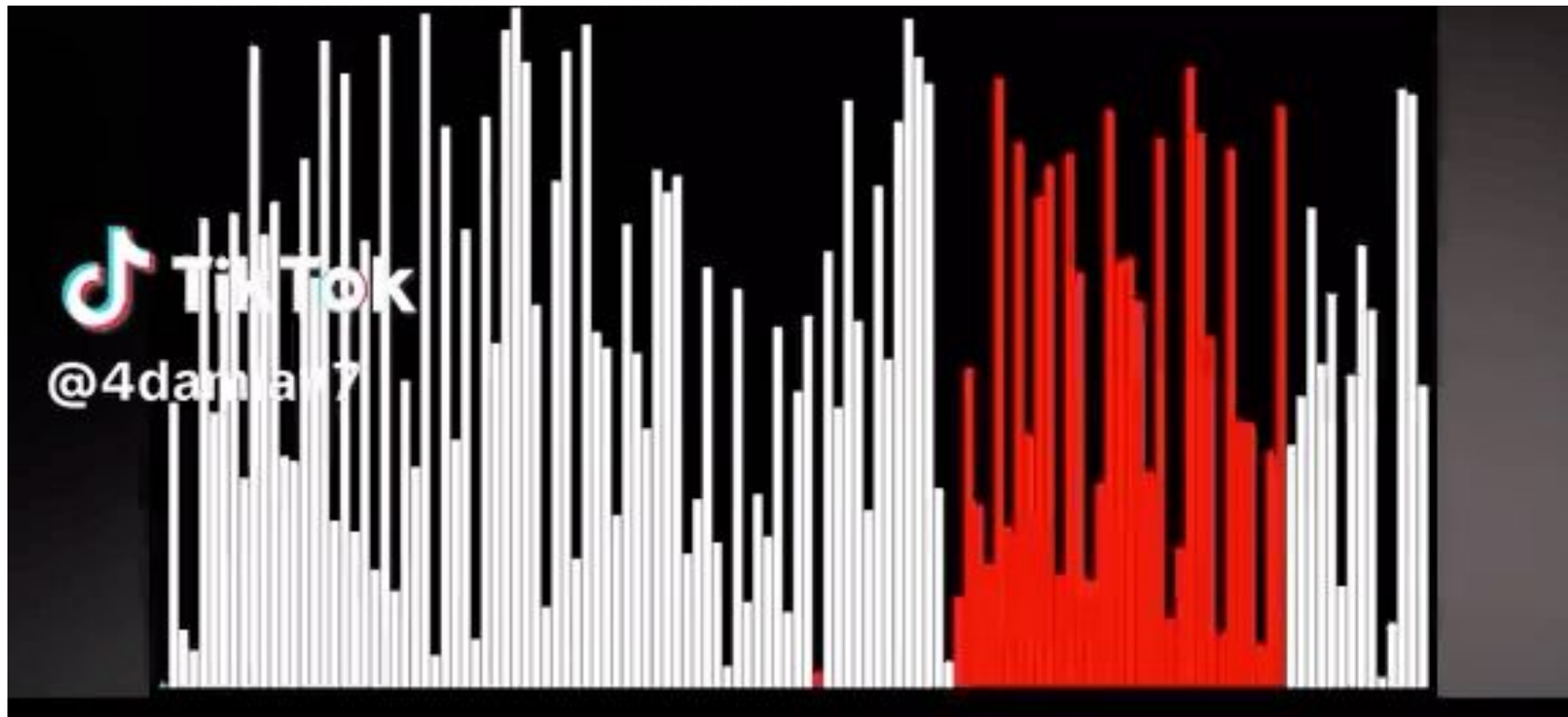


# Task Description



There are some numbers that need to be sorted...

# Task 1: Sorting Algorithm A



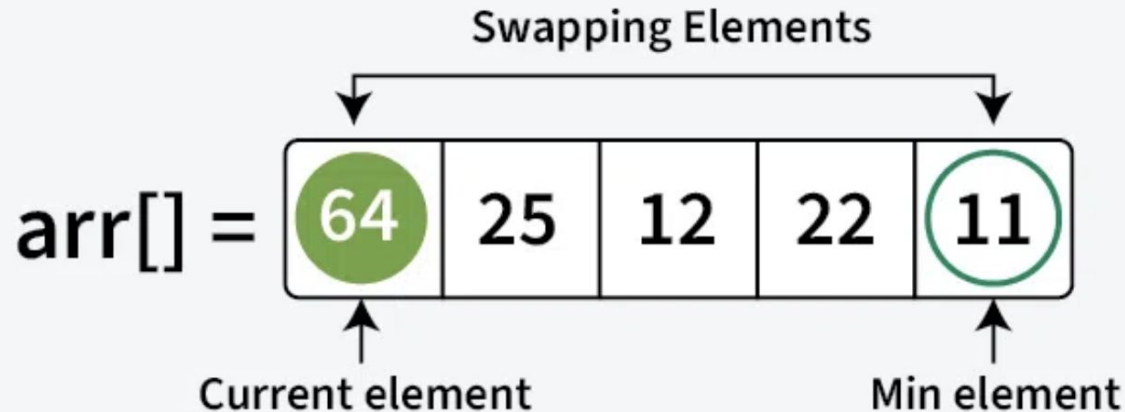
Describe the steps of this sorting algorithm.



# Task 1: Sorting Algorithm A: Selection Sort

**01**  
Step

Start from the first element at index 0, find the smallest element in the rest of the array which is unsorted, and swap (11) with current element(64).



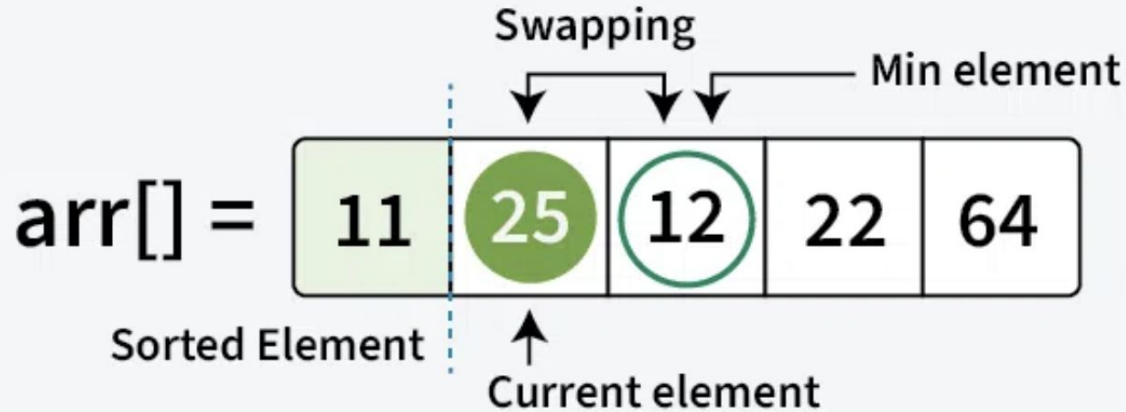
Selection Sort Algorithm



# Task 1: Sorting Algorithm A: Selection Sort

**02**  
Step

Move to the next element at index 1 (25). Find the smallest in unsorted subarray, and swap (12) with current element (25).



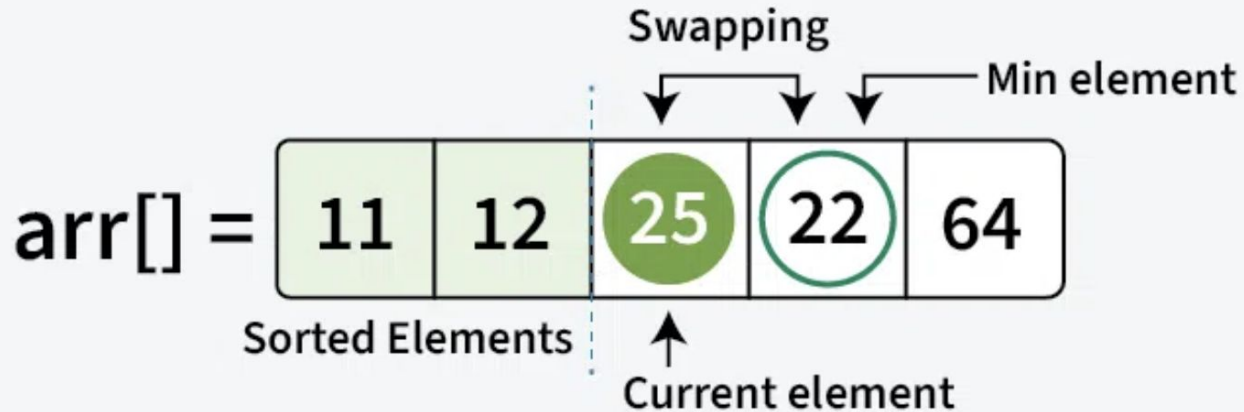
Selection Sort Algorithm



# Task 1: Sorting Algorithm A: Selection Sort

**03**  
Step

Move to element at index 2 (25). Find the minimum element from unsorted subarray, Swap (22) with current element (25).



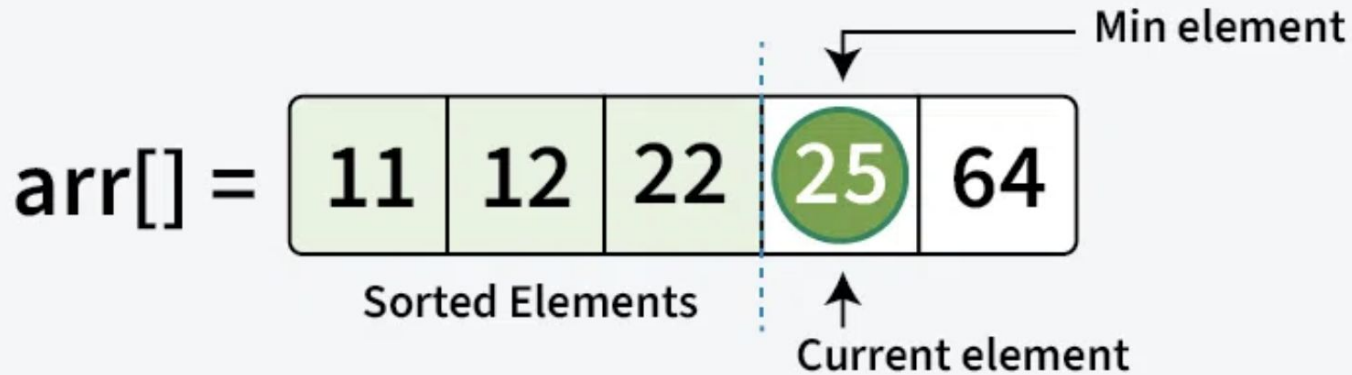
Selection Sort Algorithm



# Task 1: Sorting Algorithm A: Selection Sort

**04**  
Step

Move to element at index 3 (25), find the minimum from unsorted subarray and swap (25) with current element (25).



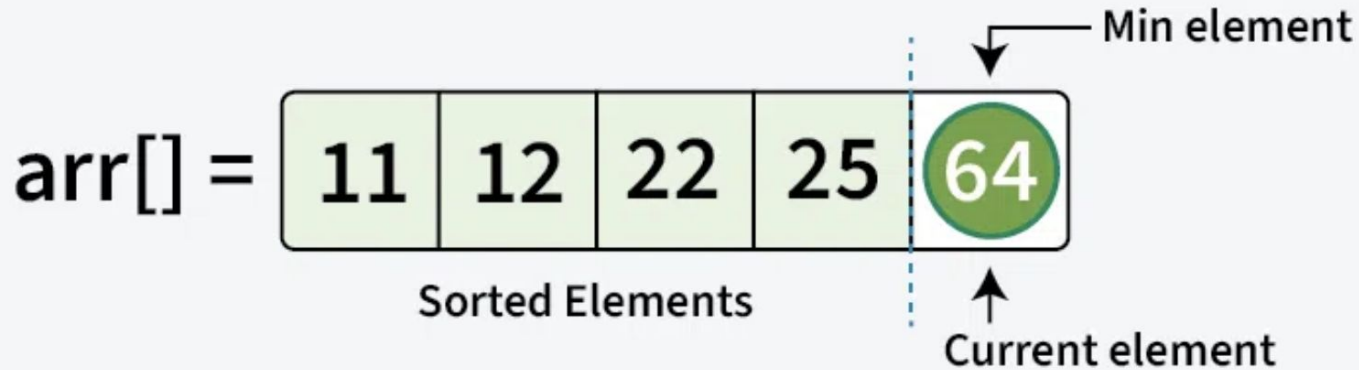
Selection Sort Algorithm



# Task 1: Sorting Algorithm A: Selection Sort

**05**  
Step

Move to element at index 4 (64), find the minimum from unsorted subarray and swap (64) with current element (64).



Selection Sort Algorithm



# Task 1: Sorting Algorithm A: Selection Sort

**06**  
Step

We get the sorted array at the end.

arr[] =

|    |    |    |    |    |
|----|----|----|----|----|
| 11 | 12 | 22 | 25 | 64 |
|----|----|----|----|----|

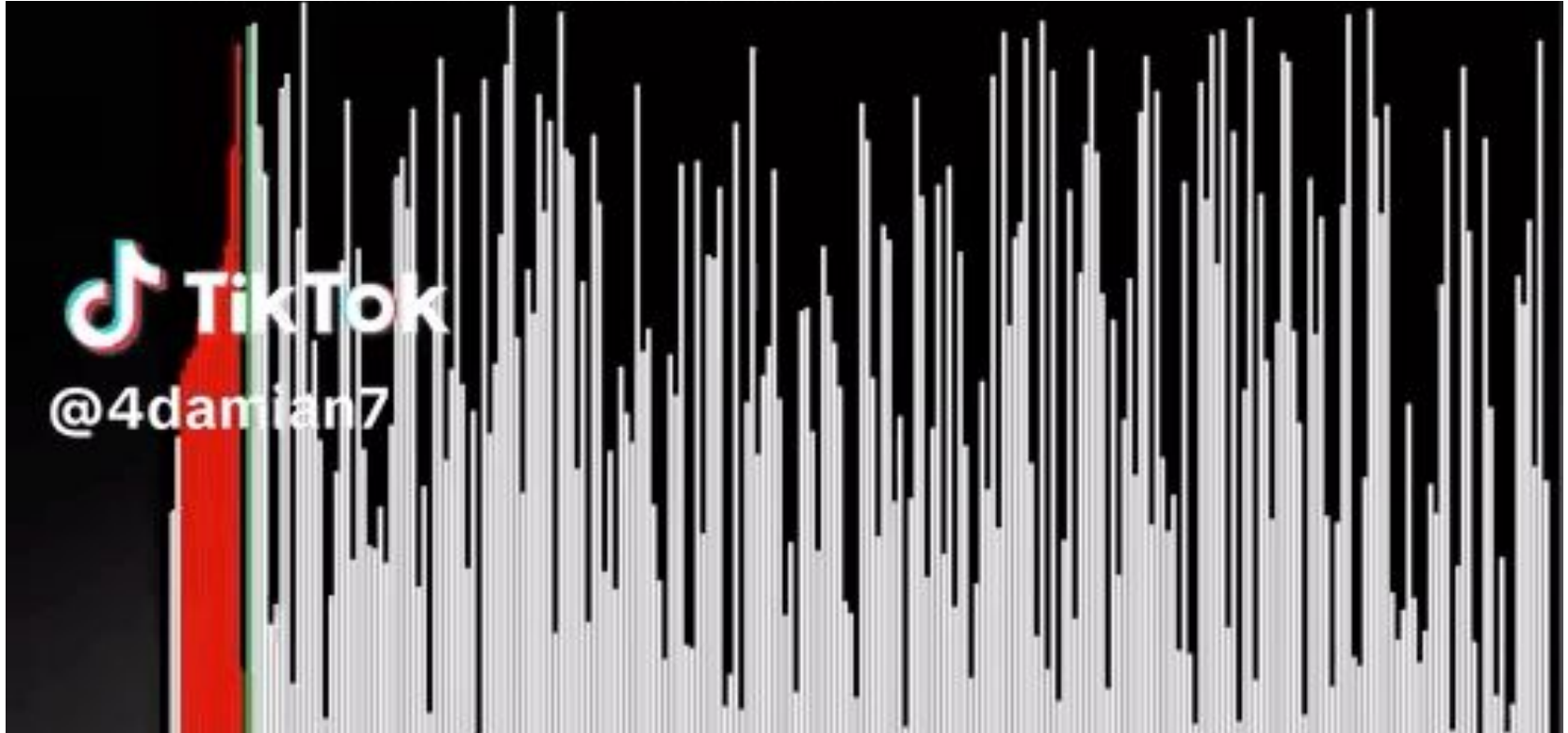
Sorted array

---

Selection Sort Algorithm



## Task 2: Sorting Algorithm B



Describe the steps of this sorting algorithm.

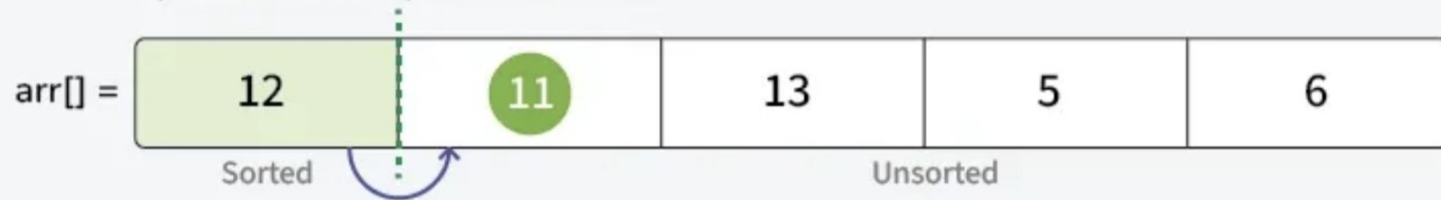


# Task 1: Sorting Algorithm B: Insertion Sort

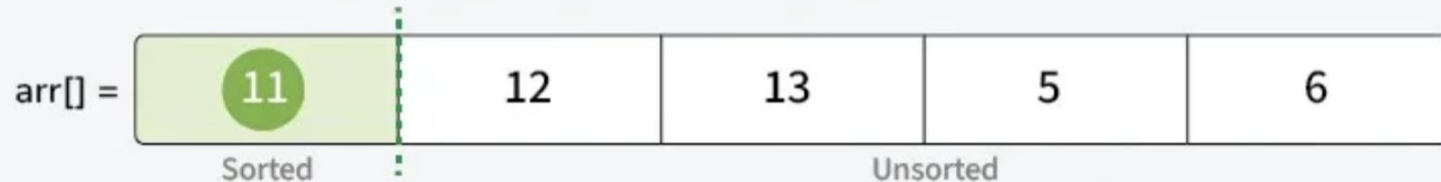
1<sup>st</sup> Pass

Key = 11

Compare key with 12. As 12 is in the wrong order, shift 12 to the right to make space for the key.



No element left to compare so insert the key at the beginning.



Insertion Sort



# Task 1: Sorting Algorithm B: Insertion Sort

2<sup>nd</sup> Pass

Key = 13

Compare key with 12, 12 is in the correct order, so no changes take place.



Insertion Sort

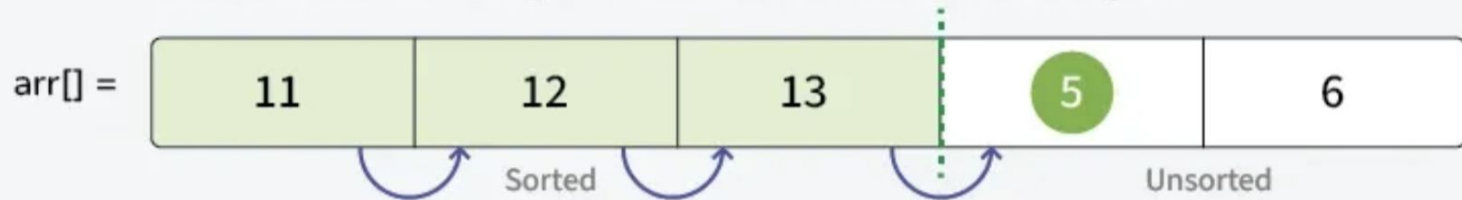


# Task 1: Sorting Algorithm B: Insertion Sort

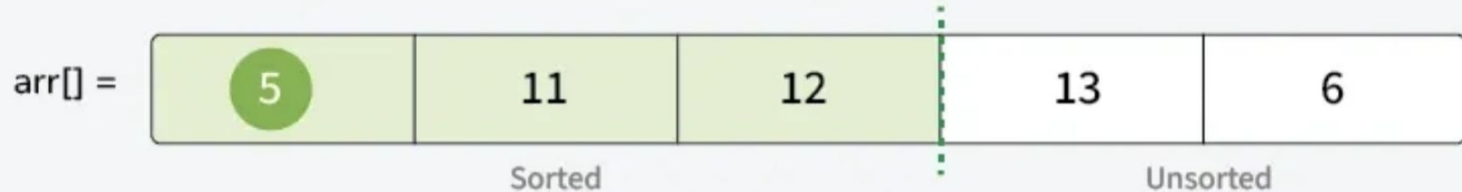
3<sup>rd</sup> Pass

Key = 5

Compare key with all the elements in the sorted subarray starting with 13, if the element is in the wrong order, shift that element to the right.



No element left to compare, so insert key at the beginning.



Insertion Sort

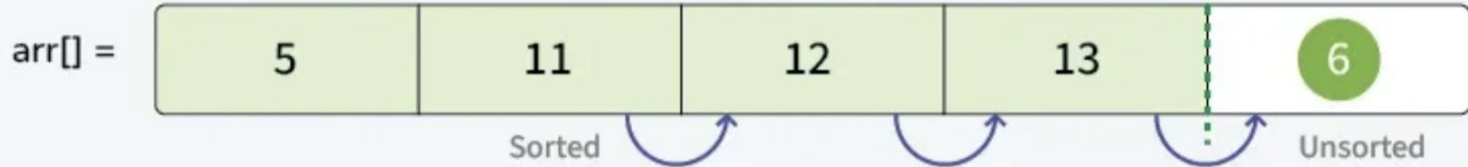


# Task 1: Sorting Algorithm B: Insertion Sort

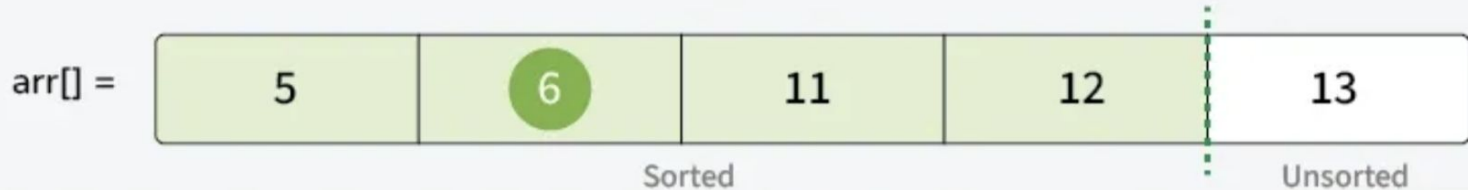
4<sup>th</sup> Pass

Key = 6

Compare key with all the elements in the sorted subarray starting with 13, if the element is in the wrong order, shift that element to the right.



5 is in the correct order, so shifting stops. Insert key after 5.



Insertion Sort



# Task 1: Sorting Algorithm B: Insertion Sort

## Sorted Array

The sorted part contains the whole array. Means that the whole array is sorted.

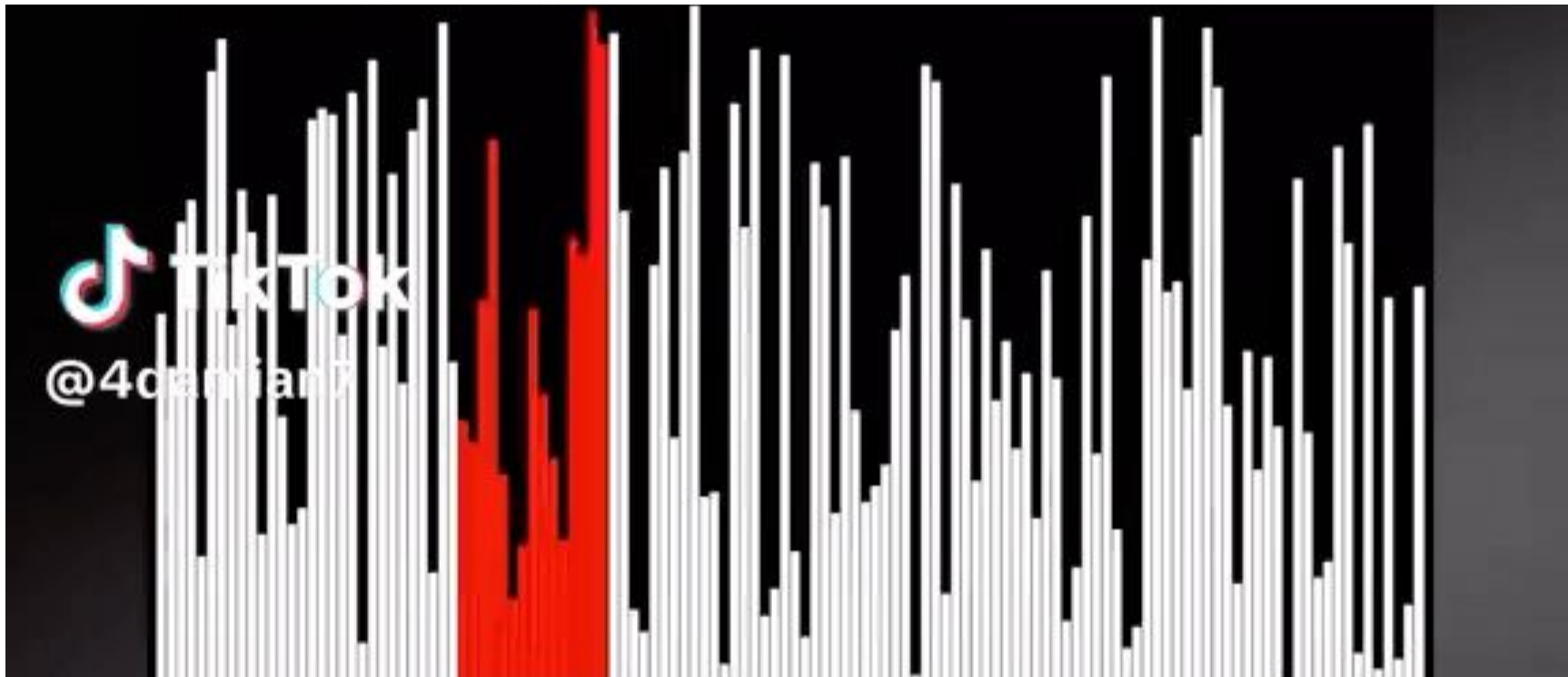
arr[] =

|   |   |    |    |    |
|---|---|----|----|----|
| 5 | 6 | 11 | 12 | 13 |
|---|---|----|----|----|

Sorted

Insertion Sort

# Task 3: Sorting Algorithm C



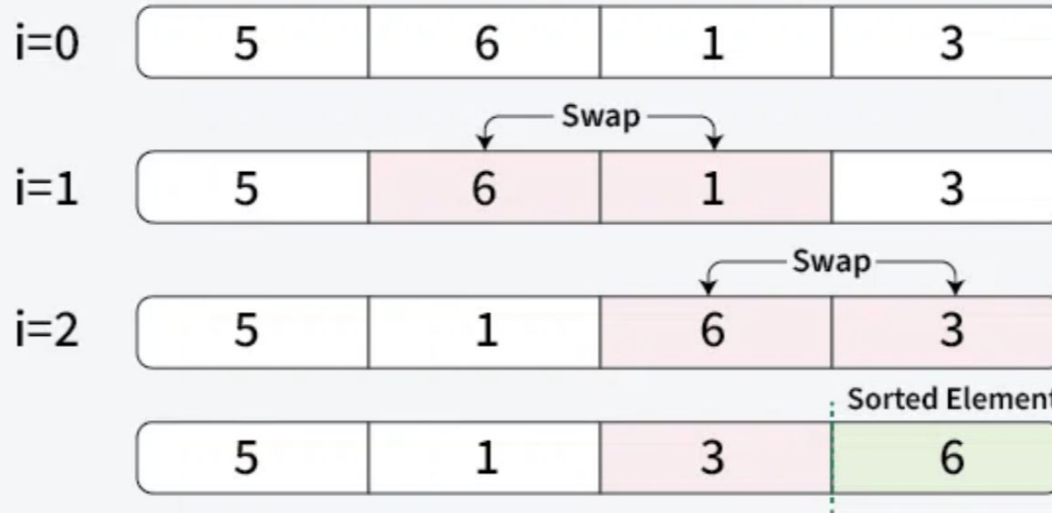
Describe the steps of this sorting algorithm (Challenge).



# Task 3: Sorting Algorithm C: Bubble Sort

**01**  
Step

Placing the 1st largest element at its correct position



Bubble sort

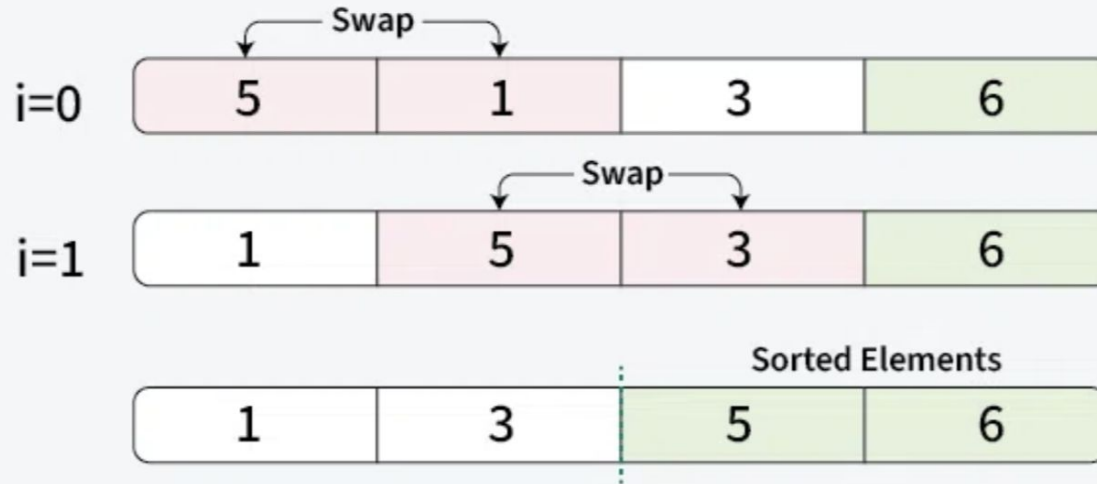




# Task 3: Sorting Algorithm C: Bubble Sort

**02**  
Step

Placing 2nd largest element at its correct position



Bubble sort



# Task 3: Sorting Algorithm C: Bubble Sort

**03**  
Step

Placing 3rd largest element at its correct position



Bubble sort



# Wrap up

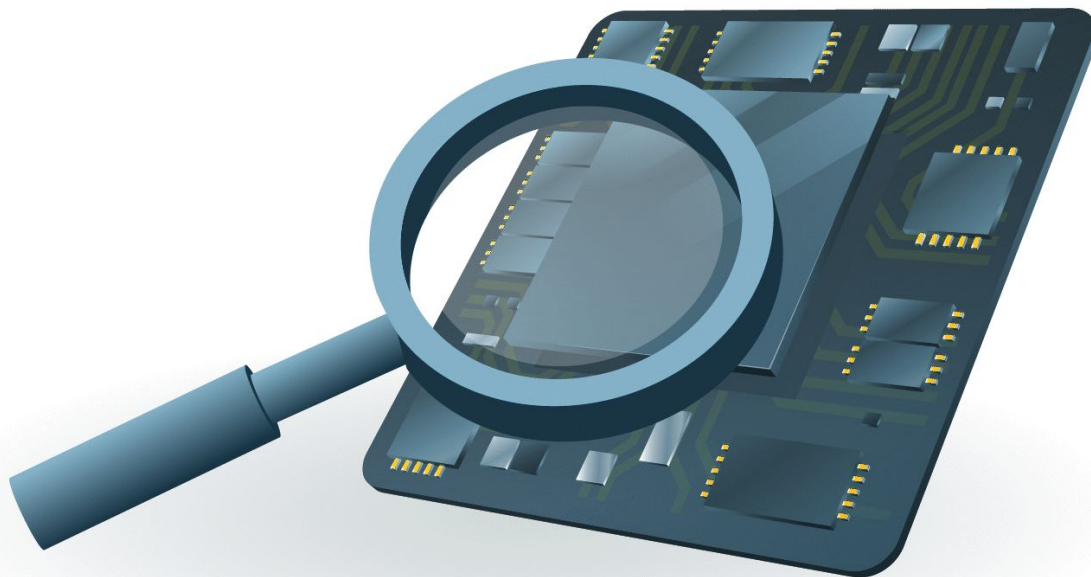


# Wrap Up

- Review the different algorithms we discussed in class today!
- Labs will begin starting next week!
  - Find ICCS X050 **before Wednesday**
- Join Ed Discussion board (link on canvas)

**Read before  
next class**

# Transistors



A transistor is a building block of modern electronics. Tens of billions of transistors are jammed into a “chip” so that data can be stored and processed. [Read more.](#)

# Moore's Law



# Moore's Law

**Gordon Moore** one of the co-founders of **Intel** corporation established the term "Moore's Law" in 1965, This law explains how the number of transistors on integrated circuits is increasing exponentially, which boosts computing capability and lowers prices. Moore's law has had a significant and far-reaching impact on technology, from the introduction of personal computers and smartphones to the advancement of artificial intelligence and Internet of Things(IoT) devices. it has accelerated development in sectors including telecommunications, healthcare, transportation, enabling organizations and people to accomplish things that were previously unthinkable.

## Moore's Law Definition

The exponential increase in the number of transistors on integrated circuits over time is referred to as Moore's law. According to this, a chip transistor count tends to double every two years or so, resulting in higher processing power and better performance.

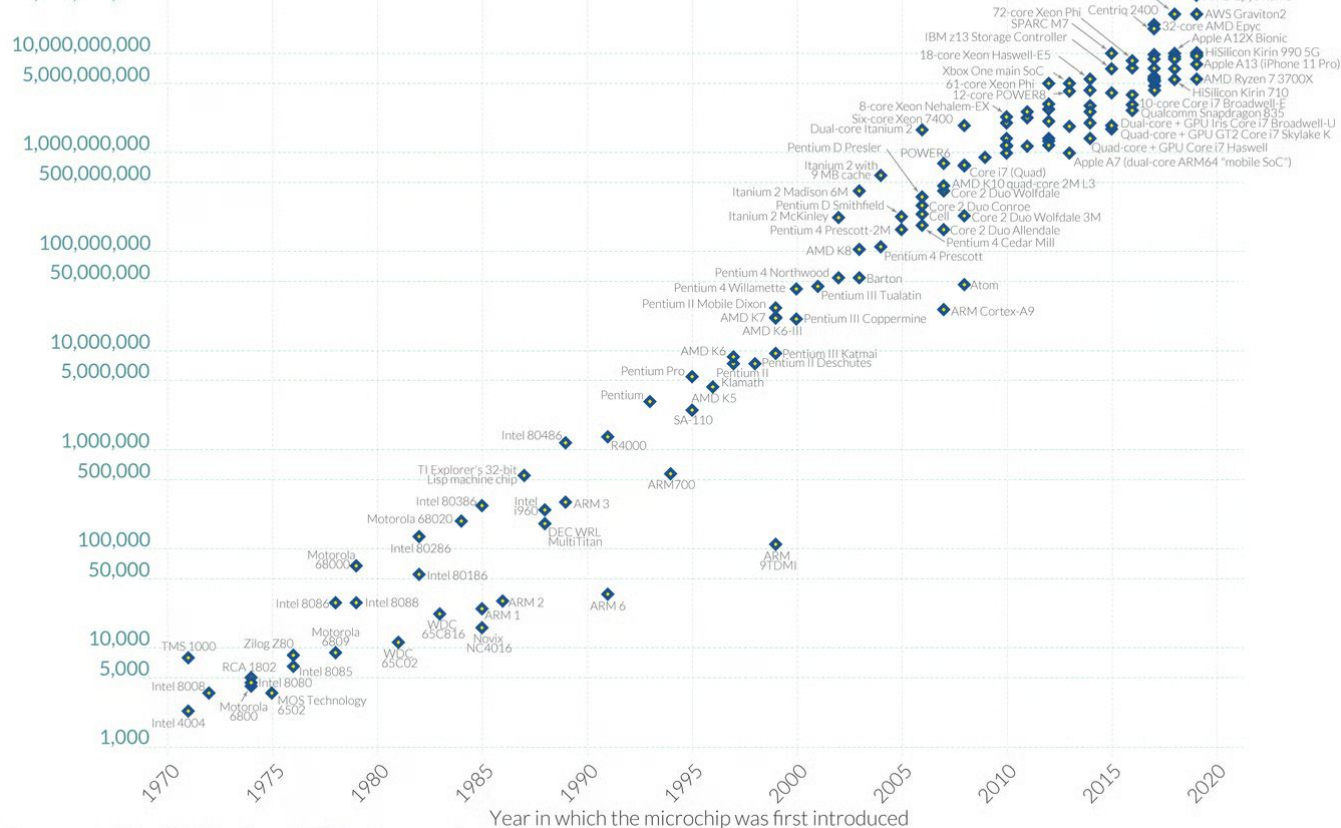
Moore's Law: The number of transistors on microchips has doubled every two years

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.

Our World  
in Data

## Transistor count

50,000,000,000



Data source: Wikipedia ([wikipedia.org/wiki/Transistor\\_count](https://wikipedia.org/wiki/Transistor_count))

OurWorldinData.org – Research and data to make progress against the world's largest problems.

Licensed under CC-BY by the authors Hannah Ritchie and Max Roser.